

CMP2204 Term Project Report

Team: git commit -m

Division of Workload

- **Yiğit Erel (2483739):** Handled the core backend daemon (p2p_file_sharing.py). Responsible for the UDP network discovery protocol, TCP file transfer routing, Diffie-Hellman key exchange logic, and DES encryption/decryption using pyDes.
- **Mehmet Eren Ekşi (2364932):** Handled the frontend architecture (gui.py). Responsible for designing the dark-themed user interface using customtkinter, seamlessly hooking the frontend to the backend using threading to prevent UI freezing, and implementing the dynamic treeview for network content discovery.

Development Platform

The application was developed and tested in a cross-platform environment using Python 3.x. Development occurred on both macOS and Windows, ensuring the customtkinter interface rendered correctly across different DPIs and that socket behaviors remained consistent regardless of the underlying operating system.

Challenges Faced

1. **Concurrency and UI Blocking:** A major challenge was the graphical interface freezing during file transfers or peer discovery. Because socket listening is a blocking operation, we implemented a strict multi-threaded architecture. All backend tasks (announcing, discovering, uploading, downloading, and wiping) were isolated into separate daemon threads so the tkinter mainloop could run uninterrupted.
2. **Cryptographic Formatting and Padding:** Establishing a secure tunnel using Diffie-Hellman and DES required strict formatting. pyDes requires exactly an 8-byte key. We overcame ValueError exceptions by strictly formatting the Diffie-Hellman shared secret (zero-filling and truncating it). Additionally, transmitting binary data securely required wrapping all chunks in Base64 encoding to ensure the JSON payloads wouldn't break during TCP transmission.
3. **Socket Resource Management:** Handling abrupt peer disconnections without crashing

the application was challenging. We resolved this by implementing strict `.setTimeout()` rules (e.g., 300 seconds for secure exchanges) and using `try/except` blocks to gracefully close sockets.

Note on Wireshark Captures

To fulfill both the assignment requirement to send our team names and the requirement to capture a PNG header, we wrote our names on an image file (`team.png`). As demonstrated in our unencrypted Wireshark capture, our backend design safely Base64-encodes all payload data to protect the JSON formatting. Therefore, the screenshot successfully demonstrates the Base64-encoded PNG magic bytes (`iVBORw0KGgo`) being transmitted.